DSMP LAB / TORONTO METROPOLITAN UNIVERSITY

DSMP Social Network Simulator User Guide

VERSION v3.2

Beginner guide for setup, datasets, MLP execution behavior, performance measurement, and testing.

Release details

DSMP LAB

Software	DSMP Social Network Simulator
Branch / release	v3.2
Repository	github.com/aabhari/test/tree/v3.2
Lab	DSMP Lab, Toronto Metropolitan University
Director	Dr. Abdolreza Abhari
Document version	Final testing guide - August 2026

Testing priorities

- Verify the normal dataset and Performance Measurement workflow.
- Distinguish the two MLP paths: Legacy MLP is uncapped in v3.2, while Sparse MLP is epoch-bounded.
- Data balancing is not available in public v3.2; use the v2.6 Dataset Import Lab when balancing is required.

Contents

1. Purpose and v3.2 Overview

DSMP is a Java multi-agent social-network simulation and recommender-system research tool with a graphical interface. It is used to load social-network-style datasets, run recommendation and machine-learning experiments, and compare accuracy and execution time.

This guide is written for a user who has not used DSMP before. It follows the same beginner-friendly structure as the v2.6 guide, but the branch and testing instructions are updated for v3.2, with special attention to MLP execution behavior and the correct availability of data-balancing features.

Version note: The public GitHub branch is named v3.2. The README displayed on that branch still carries the heading 'DSMP Simulator Version 3.1', so testers should use the Git branch indicator - not the README title - to confirm they are actually testing v3.2.

2. What to Test in v3.2

Area	What to verify	Why it matters
Data balancing availability	Confirm that the public v3.2 branch does not expose a balancing control. File -> Dataset From Text is only a file-selection dialog.	Prevents testers from looking for a control that does not exist in v3.2 and corrects the earlier documentation error.
Dataset loading	Existing DSMP datasets load correctly through File -> Dataset From Text. Direct loading does not perform preprocessing or balancing.	Separates normal dataset loading from the v2.6 import-time balancing workflow.
Performance Measurement	Get Users -> Initialize -> Run Simulation still works.	Confirms the normal experiment workflow was not broken.
Algorithms	Test both Legacy MLP and Sparse MLP when relevant. Legacy MLP may run for a long time; Sparse MLP is epoch-bounded.	Confirms the two v3.2 MLP execution paths are not confused with one another.
Accuracy reporting	Accuracy/result values are produced for completed runs using the selected algorithm and dataset.	Needed for meaningful experiment comparison.
Runtime	Record simulation/training time and note which MLP implementation was used.	Legacy MLP can consume substantial CPU because v3.2 does not cap its iterations.
Reproducibility	Repeat a run with the same dataset and settings when the chosen algorithm is deterministic or exposes a seed.	Unexpected differences may reveal a reproducibility issue.
Multi-node	Re-initialize and compare 1-node vs multi-node runs when required.	Confirms parallel experiments still operate.

3. Requirements

The v3.2 branch README specifies the same build prerequisites used by the recent DSMP branches:

- Git
- Python 3.9 or 3.10
- JDK 8 or higher (JDK 8 preferred for compatibility)
- Apache Maven

```
git --version
python3.9 --version   # or python3.10 / py -3.9
java -version
javac -version
```

```
mvn -version
```

4. Install and Start v3.2

4.1 Clone and select the correct branch

```
git clone https://github.com/aabhari/test.git
cd test
git checkout v3.2
git branch --show-current   # expected: v3.2
```

4.2 First-time setup

```
Windows:      py -3.9 build.py setup
macOS/Linux:  python3.9 build.py setup
```

The repository build script creates/updates the Python virtual environment, installs Python and Java dependencies, downloads required NLTK data, and compiles the Java source.

4.3 Start the simulator

```
Windows:      py -3.9 build.py run
macOS/Linux:  python3.9 build.py run
```

Keep the terminal open. It is useful for seeing progress, warnings, and exceptions that may not be obvious in the GUI.

5. DSMP Input Data Format

The existing DSMP workflow uses a UTF-8, TAB-delimited text representation. The v2.6 testing guide documents the common six-column follower-followee format as:

```
followee<TAB>postId<TAB>yyyy-MM-dd[ HH:mm[:ss]]<TAB>userId<TAB>userName<TAB>text
```

Column	Meaning
followee / reference user	Class or recommendation target.
postId	Numeric record/tweet/post ID.
date	Record date; use a DSMP-compatible date format.
userId	Numeric user/customer/author ID.
userName	Name/label of the entity producing the text.
text	Tweet, product description, paper title, or other text content.
Important: In v3.2, File -> Dataset From Text loads a DSMP-ready text file only. It does not provide preprocessing or class-balancing settings. If balancing is required, create the balanced DSMP file in the v2.6 Dataset Import Lab first, then load that output file into v3.2.	

6. Loading and Preparing a Dataset

1. Start DSMP and switch to Performance Measurement.

2. Open File -> Dataset From Text.

3. Choose the DSMP-ready dataset to test.

4. Set the date range so it includes the records in the file.

5. Set Number of Nodes = 1 for the first baseline.

6. Choose the algorithm and other experiment settings.

7. Record the dataset name, number of records/users/classes if shown, and the selected algorithm/model settings before the run.

Recommended first datasets

Use a small known dataset for the first smoke test. For Legacy MLP, begin with a small dataset because training can be computationally expensive. After the small test passes, repeat with the larger retail dataset requested by the research team if needed.

7. Data Balancing Availability in v3.2 and v2.6 Workflow

Important correction: Data balancing is not present in the public v3.2 implementation. Section 7.2 of the earlier guide incorrectly described a v3.2 balancing control. File -> Dataset From Text is only a file-selection dialog and does not contain preprocessing or balancing settings.

7.1 What is available in v3.2

1. **Load an existing DSMP-ready file through File -> Dataset From Text.**
2. **Treat the loaded file as already prepared; v3.2 does not apply class balancing during this step.**
3. **Continue with the normal Performance Measurement sequence: Get Users -> Initialize -> Run Simulation.**
4. **Record the dataset, algorithm/model settings, accuracy/result, and runtime for the run.**
5. **Do not report the absence of a balancing control in v3.2 as a software bug; it is expected behavior for this branch.**

7.2 Data balancing in v2.6

1. **In v2.6, open User Generation Simulation -> Dataset Import Lab.**
2. **Select Inspect dataset, then open 5. Class Balance.**
3. **Select the balance mode, target policy, optional target size, and random seed required for the test.**
4. **Select Refresh Diagnosis to review the expected before-and-after class distribution.**
5. **Select Build DSMP Dataset, then Use Output in Simulator when you are ready to use the generated file.**
6. **The tool creates a new balanced DSMP file together with metadata and an audit report; it does not modify the original dataset.**

7.3 Balancing an existing DSMP text file in v2.6

Step	Tester action
Load source	Open Dataset Import Lab and choose Other / custom DSMP mapping.
Map columns	Map the six TAB-delimited DSMP columns: followee, postId, date, userId, userName, and text.
Inspect balance	Open 5. Class Balance and select the required balance mode and target policy.
Set options	Enter an optional target size and random seed when required.
Diagnose	Use Refresh Diagnosis to review the expected before-and-after distribution.
Build output	Select Build DSMP Dataset. The original input file remains unchanged.
Use output	Select Use Output in Simulator, or load the generated balanced DSMP file later in v3.2.

7.4 Using a balanced dataset with v3.2

1. **Create the balanced DSMP output in v2.6 Dataset Import Lab or another approved preprocessing workflow.**

2. Start v3.2 and open Performance Measurement.

3. Load the balanced DSMP output through File -> Dataset From Text.

4. Click Get Users, then Initialize, then Run Simulation.

5. Record the result and runtime, and identify the loaded file as pre-balanced input.

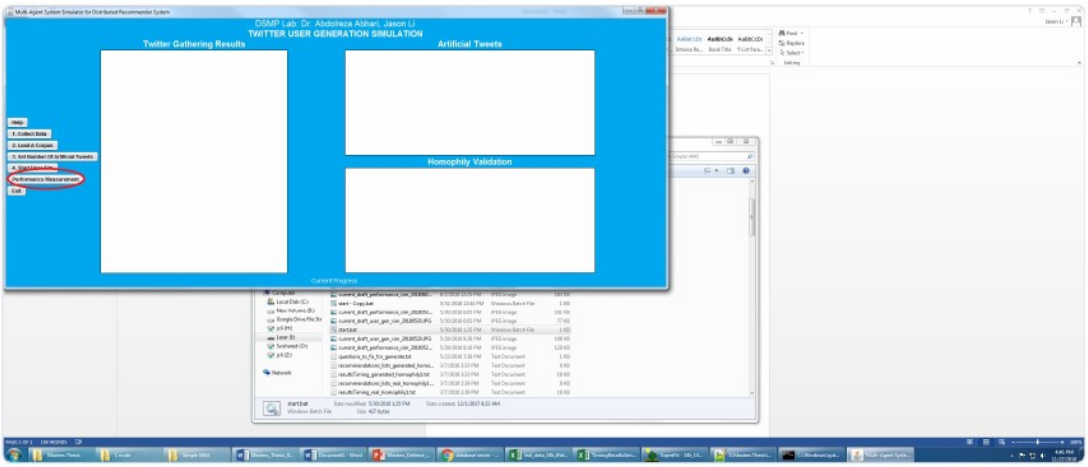
6. When comparing with the original file, keep the same algorithm, node count, date range, and model settings.

7.5 Common documentation/testing mistakes

- Looking for a balancing control in public v3.2. It is not available in that branch.
- Expecting File -> Dataset From Text to contain preprocessing or balancing settings. It only selects a file.
- Loading a DSMP text file directly in v3.2 and assuming it has been balanced automatically.
- Describing a missing v3.2 balancing control as a defect rather than a documentation discrepancy.
- Overwriting the original source file instead of creating a new balanced output in the v2.6 Dataset Import Lab.
- Comparing original and pre-balanced runs while changing algorithm/model settings at the same time.
- Failing to record the v2.6 balancing configuration, diagnosis, metadata, or audit report used to create the output.
- Using balancing instructions from v2.6 without clearly labeling them as v2.6-only instructions.

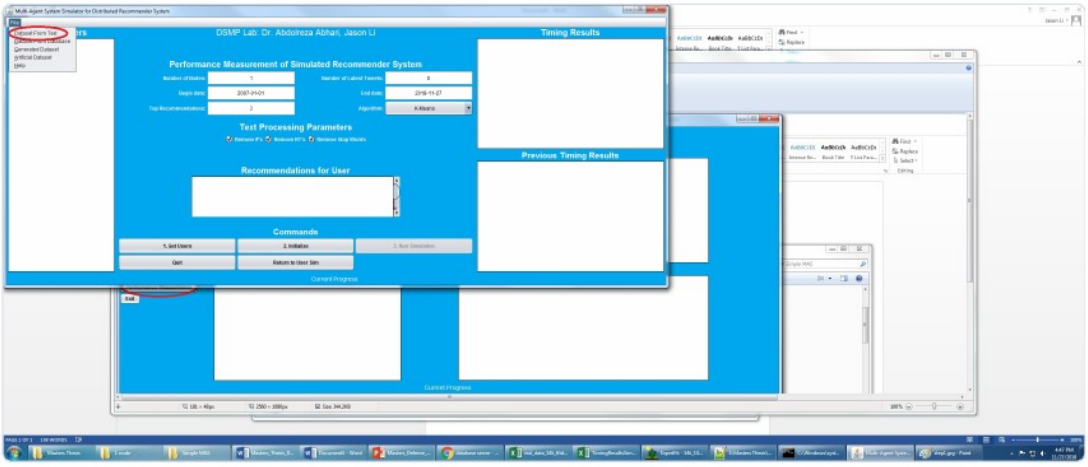
8. Performance Measurement - Beginner Walkthrough

The core Performance Measurement sequence is the same sequence documented in the v2.6 guide: switch modes, load a text dataset, Get Users, Initialize, and Run Simulation. The screenshots below are retained as orientation images; button placement may differ slightly in v3.2.



1. On the User Generation screen, click **Performance Measurement**.

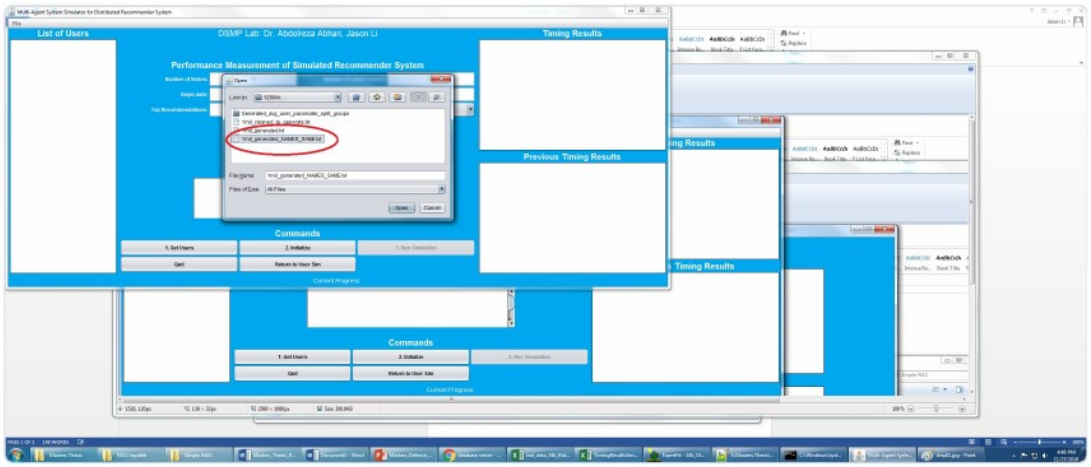
Figure 2 — Open a dataset from text



1. In Performance mode, open **File → Dataset From Text**.

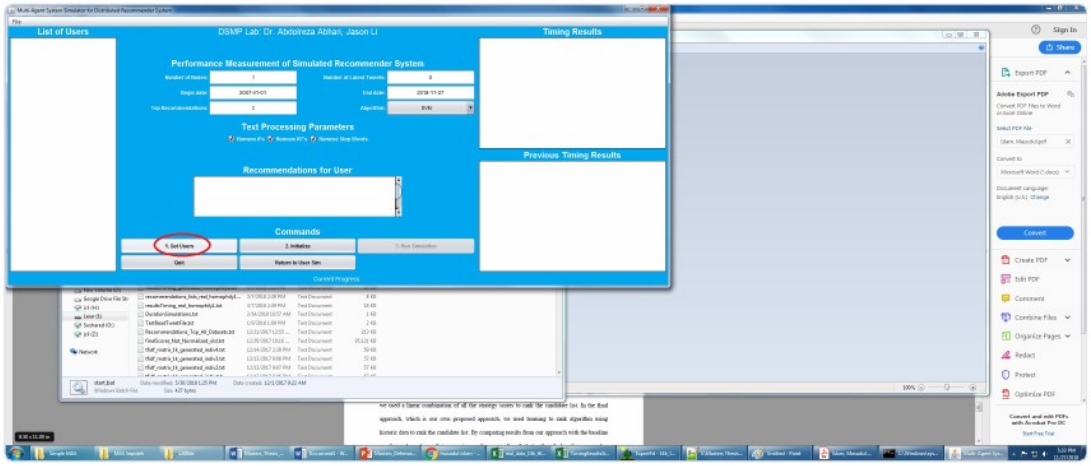
Figure 3 — Choose a corpus file

Figure 1. DSMP workflow reference image retained from the earlier user-guide screenshot set; use it to identify the core Performance Measurement sequence in v3.2.



1. Select a ready DSMP file, for example:
2. `Dataset/Reduced_14k.txt` (quick Twitter check)
3. `Dataset/Retail.txt` (MLP retail test)
4. or your newly imported file from Section 6

Figure 4 — Set parameters, then Get Users



1. Set parameters, then click **1. Get Users**.

Suggested first-run values:

Figure 2. DSMP workflow reference image retained from the earlier user-guide screenshot set; use it to identify the core Performance Measurement sequence in v3.2.

8.1 Standard run sequence

1. Click Performance Measurement.

2. Open File -> Dataset From Text and select the dataset.

3. Set Number of Nodes, date range, Top Recommendations, algorithm, and text-processing options.

4. Click 1. Get Users.

5. Click 2. Initialize.

6. Click 3. Run Simulation.

7. Review recommendations, accuracy/algorithm messages, timing results, and terminal output.

Important: After changing the dataset, algorithm parameters, or node count, initialize again before the next run. Loading a different file - including a file that was pre-balanced in v2.6 - counts as changing the dataset.

9. Running Machine-Learning Experiments

The v3.2 repository is a DSMP research branch, so the safest test strategy is to compare one controlled variable at a time. Do not change the dataset, algorithm settings, and node count simultaneously. If comparing an original file with a pre-balanced file created in v2.6, keep the v3.2 run settings identical.

Experiment	Keep fixed	Change
Dataset comparison	Algorithm, node count, date range, model settings	Original file vs pre-balanced file
Algorithm comparison	Dataset and node count	Algorithm only (for example Legacy MLP vs Sparse MLP)
Parallel comparison	Dataset and algorithm/model settings	Node count only
Repeatability	Everything, including MLP implementation	Nothing; repeat same run/seed

MLP execution-time note: The default MLP uses the Legacy Neuroph engine. On a full dataset it can be computationally expensive. In v3.2, Legacy MLP training has no configurable maximum-iteration limit, so it continues until it reaches the error threshold. This can consume substantial CPU and make the interface appear frozen even though training is still running. v3.2 also includes Sparse MLP, which follows an epoch-bounded training path and should be distinguished from Legacy MLP when documenting runtime behavior. In v2.6, Legacy MLP can be bounded by selecting MLP, opening MLP Settings, enabling custom settings, and entering a positive Max iterations value; a value of 0 preserves unlimited behavior. The Fast bounded option in MLP Auto Search is also suitable for preliminary testing.

10. Comparing Original vs. Pre-Balanced Results

Field	Original run	Pre-balanced run
Dataset / file	Original file	Pre-balanced file
Total records	_____	_____
Class distribution	_____	_____
Balancing source/settings	Not applied	v2.6 Dataset Import Lab / _____
Random seed, if used to balance	N/A	_____
Algorithm	_____	same
Nodes	_____	same
Model parameters	_____	same
Accuracy / score	_____	_____
Runtime	_____	_____
Completed?	Yes / No	Yes / No
Notes / anomalies	_____	_____

A v3.2 comparison can use an original DSMP file and a separate balanced DSMP file created in v2.6 Dataset Import Lab. v3.2 itself does not perform the balancing step. For a fair comparison, keep the algorithm, node count, date range, and model settings the same and record how the balanced input was produced.

11. User Generation Simulation

The User Generation mode can still be used when the experiment requires artificial/generated social-network data. In public v3.2, however, it does not provide the v2.6 Dataset Import Lab balancing workflow described in Section 7. If class balancing is required, prepare the balanced DSMP output in v2.6 first and then load that file into v3.2.

1. Load or prepare the corpus.

2. Set the requested artificial-tweet/user-generation parameters.

3. Generate the data.

4. Load the generated DSMP file into Performance Measurement.

5. If a pre-balanced input is used, document that it was created outside v3.2 and record the v2.6 balancing settings/audit information when applicable.

12. Multi-node / Parallel Tests

1. Complete a correct 1-node run first.

2. Change Number of Nodes to the requested value (for example 2, 4, or 8).

3. Re-initialize.

4. Run the same dataset and algorithm/model configuration.

5. Compare timing and accuracy with the 1-node baseline.

Important: Complete a correct 1-node run before using a multi-node result for diagnosis. For MLP tests, also record whether the run used Legacy MLP or Sparse MLP.

13. Recommended v3.2 Validation Matrix

Test	Dataset / setting	Pass condition
A. Startup	Fresh v3.2 checkout	Setup and run complete; GUI opens.
B. Existing data	Small DSMP-ready file	Get Users -> Initialize -> Run completes.
C. Legacy MLP smoke test	Small dataset; Legacy MLP	Training starts and progress/CPU activity can be observed; long runtime is not automatically treated as a freeze.
D. Sparse MLP test	Small dataset; Sparse MLP	Run uses the epoch-bounded Sparse MLP path and completes according to its configured training behavior.
E. Balancing availability	Public v3.2 UI	No v3.2 balancing control is expected; File -> Dataset From Text remains file selection only.
F. Pre-balanced input	Balanced DSMP file produced in v2.6	Balanced file loads and runs in v3.2 as an already-prepared dataset.
G. Large retail	Shared large retail file	Use caution with Legacy MLP because uncapped training may consume substantial CPU/time.
H. Multi-node	Same dataset/algorithm; >1 node	Run completes after re-initialization.
I. Regression	Previously working DSMP dataset	Normal non-balancing workflow still works.

14. Troubleshooting

Problem	What to try
Wrong branch	Run git branch --show-current and confirm v3.2.
README says Version 3.1	Use the Git branch indicator; the v3.2 branch currently displays an older README heading.
Python error	Use Python 3.9 or 3.10.
java/javac/mvn not found	Install JDK/Maven and add them to PATH.
Setup/build fails	Run python build.py info, then retry with -v.
Stale build	Run python build.py clean followed by python build.py setup.
No users loaded	Check DSMP file format and Begin/End dates.
Balancing control not found	Expected in public v3.2. Use v2.6: User Generation Simulation -> Dataset Import Lab -> Inspect dataset -> 5. Class Balance.
Dataset From Text has no balancing settings	Expected. File -> Dataset From Text is only a file-selection dialog and bypasses balancing.
Legacy MLP looks frozen	Check terminal/log output and CPU activity. In v3.2 Legacy MLP has no configurable max-iteration cap and may still be training until the error threshold is reached.
Results cannot be compared	Re-run with identical dataset role, algorithm, node count, date range, and model parameters; record Legacy vs Sparse MLP explicitly.

15. Bug Report Template

Use this format when a DSMP tester finds a possible v3.2 issue:

Field	Record
Branch / commit	v3.2 / _____
Operating system	_____
Dataset	_____
Record/user/class counts	_____
MLP implementation	Legacy MLP / Sparse MLP / N/A
MLP / algorithm settings	_____
Date range / text-processing settings	_____
Number of nodes	_____
Steps to reproduce	1. ... 2. ... 3. ...
Expected result	_____
Actual result	_____
Error/log message	_____
Reproducible?	Always / Sometimes / Once
Screenshot/result file	_____

16. Development / Repository Rules

Follow the lab's incremental/sprint-style process for simulator changes. Keep software modifications controlled, unit-test them, prepare a change list, and review before merging into the shared branch. Documentation can be maintained separately without mixing unreviewed simulator changes into the active code.

- Do not push unreviewed simulator code directly to the shared active branch.
- Keep experimental fixes local or on an agreed branch until tested.
- Record the exact v3.2 commit used for every test.
- Do not overwrite the original dataset when creating processed or pre-balanced test data in another workflow.
- Attach logs, runtime information, algorithm/model settings, and the MLP implementation name to long-running or failed MLP bug reports.

17. One-page Cheat Sheet

1. **git clone the aabhari/test repository -> checkout v3.2.**
2. **Run Python 3.9/3.10 build.py setup.**
3. **Run build.py run.**
4. **Performance Measurement -> File -> Dataset From Text.**
5. **Remember: public v3.2 has no data-balancing control; Dataset From Text is file selection only.**
6. **For balancing, use v2.6 -> User Generation Simulation -> Dataset Import Lab -> Inspect dataset -> 5. Class Balance.**
7. **In v2.6, choose balance mode/target policy, optional target size and seed -> Refresh Diagnosis -> Build DSMP Dataset -> Use Output in Simulator.**
8. **Load the resulting balanced DSMP file into v3.2 if you need to test it there.**
9. **For Legacy MLP, expect potentially high CPU/long runtime in v3.2 because there is no configurable maximum-iteration limit.**
10. **Distinguish Legacy MLP from the epoch-bounded Sparse MLP when recording results.**
11. **Report bugs with branch/commit, dataset, MLP implementation, model settings, runtime/logs, and exact reproduction steps.**

Appendix A - Figure Notes

The reference screenshots in this guide come from the earlier DSMP user-guide screenshot set. They are included to help first-time users recognize the Performance Measurement workflow. Because v3.2 is a newer branch, testers should follow the controls that actually exist in the running v3.2 application. In particular, do not infer a v3.2 balancing control from older screenshots or v2.6 instructions.

Appendix B - Sources and Verification

- Public GitHub repository: aabhari/test, branch v3.2.
- v3.2 branch README/build.py instructions for prerequisites, setup, run, build-tool help, and troubleshooting.
- Existing DSMP v2.6 Combined Final User Guide supplied for document style, common DSMP data format, the beginner workflow, and the v2.6 Dataset Import Lab balancing path.
- Technical clarification from DSMP Lab (August 2026): public v3.2 does not expose data balancing; v2.6 provides import-time class balancing; v3.2 Legacy MLP is uncapped by maximum iterations, while Sparse MLP is epoch-bounded.

Repository:
`https://github.com/aabhari/test/tree/v3.2`

Verification note: Public v3.2 should not be documented as having an in-application balancing control. When a balanced dataset is required, record the v2.6 Dataset Import Lab configuration and use the generated DSMP output as prepared input. For MLP runtime reports, always identify whether the run used Legacy MLP or Sparse MLP.

End of DSMP v3.2 User Guide.